

xStore — Vendor Commission Wallet & Settlement

Full business case catalogue for backend implementation · prepared 2026-08-09

This document enumerates every business case the vendor commission wallet / settlement system must handle, so backend can design the ledger schema and endpoints against the real behavior we need — not just the happy path. It assumes the locked model already agreed with the owner:

- 100% Cash on Delivery (COD) — the app never holds buyer money for self-delivered orders.
- Commission is a platform-set % per category, deducted from the vendor, never added to the buyer's price.
- The vendor wallet is a **one-directional owed-balance ledger** (vendor owes platform), not a two-way payout account.
- Settlement is weekly; two independent thresholds (warn, pause) gate a vendor's ability to publish new listings.
- A courier-delivery pilot exists in parallel: for courier-fulfilled orders the platform collects the COD cash directly and pays the vendor net, so **no debt is created** for those orders. The wallet only ever accrues for self-delivered orders.

Each table below lists: the case, what triggers it, the expected system behavior, and — where the business rule hasn't been locked yet — the open question backend should raise before building it.

1. Order → commission accrual lifecycle

Governs how a single order's commission moves through **pending** → **due** → **settled / voided**. This is the state machine backend needs per order-item.

Case	Trigger	Expected behavior	Notes / open question
Self-delivered order, delivered	Order status → delivered, fulfilled by vendor (no courierId)	commissionStatus: pending → due. Adds commissionAmount to the vendor's owed balance.	Primary accrual path — this is the only case that increases wallet debt.
Courier-delivered order, delivered	Order status → delivered, fulfilled by platform courier	Platform already holds the cash; commission is settled at delivery, not accrued as debt. Vendor is paid net.	Must be a genuinely separate path, not “due then auto-settled” — it should never appear as owed balance, even momentarily.
Order cancelled before delivery	Order status → cancelled while commission is still pending	commissionStatus → voided. Never touches the wallet balance.	—
Order returned / refused after delivery (self-delivered)	Buyer refuses COD, or a valid return is processed post-delivery	Reverse the already-accrued commission: due → voided (reversal), balance decreases by that amount.	Not yet locked: is this a straight reversal, or a separate negative-adjustment entry so the audit trail shows the original charge and the reversal as two lines?
Partial return on a multi-item order	One line item of a multi-item order is returned; rest are kept	Only that item's commissionAmount reverses; other items in the same order stay due/settled normally.	Confirms commission must be tracked per order-item, not per order, in the schema.

Case	Trigger	Expected behavior	Notes / open question
Failed delivery attempt, later re-delivered	Delivery fails, order re-attempted, eventually delivered	Commission accrues exactly once, on the delivery that actually succeeds.	Risk: if accrual is driven off a generic “status changed to delivered” event rather than an idempotency check, a retried delivery could double-charge. Needs a guard.

2. Trust / fraud exposure

Self-delivered COD is the platform's real exposure: the vendor collects cash directly from the buyer with no independent confirmation from the platform side.

Case	Trigger	Expected behavior	Notes / open question
Vendor collects cash, then marks order cancelled / “buyer refused”	Vendor-controlled status update after actual hand-to-hand delivery	System has no way to distinguish this from a genuine refusal today.	Open decision: accept as a cost of the self-delivery fallback, or add a mitigation (buyer delivery confirmation, SMS/OTP-on-delivery, audit sampling, vendor trust scoring)? Courier-delivered orders don't have this hole because the courier — not the vendor — controls the status.

Recommend flagging this explicitly to backend/owner now — it's a policy decision, not an engineering one, and shapes whether delivery-confirmation needs a dedicated endpoint.

3. Settlement — how debt actually reaches zero

Because COD orders generate no platform-held payout to net against, settlement requires an **active payment the vendor makes to the platform**. This is the largest gap between what's designed and what's built today.

Case	Trigger	Expected behavior	Notes / open question
Vendor pays owed balance in full	Weekly settlement cycle, or vendor pays ad hoc	All due entries for that vendor → settled. Balance zeroed.	—
Vendor pays a partial amount	Vendor pays less than the full owed balance	Balance reduces by the paid amount.	Not decided: does the payment apply FIFO to the oldest due entries (so per-order statements stay accurate), or just reduce the aggregate number? Needed for vendor-facing statements to reconcile.
Vendor pays nothing	No payment received by cycle end	Balance carries forward into the next cycle.	Current implementation has no true weekly reset — balance is just “everything currently due”, checked continuously against thresholds rather than reset on a cycle boundary. Confirm this is the intended behavior before backend builds a cycle-based settlement job.
Payment channel / how cash physically moves	Vendor wants to settle their balance	Not yet defined — candidates: in-person cash handoff (mirrors the courier cash-handover model already designed), bank transfer, mobile wallet (Vodafone Cash / InstaPay).	Whichever channel is chosen, an admin/owner action is required to mark it settled — this is unbuilt (“Phase C”). Needs its own endpoint + audit trail (who marked it settled, when, evidence reference).
Vendor overpays (rounds up, double-pays old debt)	Payment received exceeds current owed balance	Not yet supported — the ledger is one-directional and has no slot for a credit balance.	Open decision: refund the excess, or carry it forward as a credit against next week? If the latter, the ledger needs to support a negative-owed state, which is a schema change from the current one-directional design.

4. Threshold enforcement (warn / pause)

Two independent, owner-configurable weekly EGP thresholds on the owed balance. Today: warn = 100 EGP, pause = 200 EGP, hardcoded client-side constants — no backend config endpoint exists yet.

Case	Trigger	Expected behavior	Notes / open question
Balance below warn threshold	Normal operation	No restriction, no notification.	—
Balance crosses warn threshold	Owed balance reaches the warn value	Vendor is notified (“pay X to avoid being paused”). Still fully orderable.	—

Case	Trigger	Expected behavior	Notes / open question
Balance crosses pause threshold	Owed balance reaches the pause value	Vendor is blocked from publishing new listings.	Gap: does NOT currently pull already-live listings offline, so a paused vendor can keep taking orders through listings that were live before the pause. This contradicts the intended “pause = stop taking new orders” behavior — needs a bulk listing status-update capability that doesn't exist yet.
Vendor pays down below pause threshold	Balance reduced via settlement (see §3)	Vendor should auto-unpause and regain publishing ability.	Confirm this reverse transition is implemented, not just the one-way trip into pause.
Vendor never pays, stays paused indefinitely	No settlement after repeated cycles	Undefined today.	Policy decision needed: escalate to account suspension? Manual collections outreach by owner/admin? At what balance / how many cycles?

5. Vendor exit / account-level cases

Case	Trigger	Expected behavior	Notes / open question
Vendor deactivated or banned with outstanding debt	Admin action or policy violation while balance > 0	Debt should not simply disappear with the account.	Needs the debt to survive as a claim against the vendor's verified identity (phone / national ID), independent of the app account's lifecycle.
Vendor deletes and re-registers to dodge debt	New account created with same real-world identity	New account should inherit / be blocked by the prior unpaid balance.	Same identity-linking mechanism as above, viewed from the abuse angle — needs dedup on registration against known debtor identities.
Vendor disputes a specific commission line	Vendor claims wrong rate applied, or disputes that an order occurred	Needs a formal adjustment path on individual ledger entries with an audit trail.	Without this, disputes become ad hoc manual DB edits with no record of who approved what and why.

6. Rate / config change cases

Case	Trigger	Expected behavior	Notes / open question
Owner changes a category's commission % mid-week	Config update to commission-config (category → percent)	Rate should apply only to orders delivered after the change; orders already accrued keep the rate that was in effect at their delivery time.	Recommend locking this explicitly: rate is fixed at order-delivery time, never retroactively recalculated. Otherwise a vendor's balance changes underneath them with no corresponding new transaction, which breaks auditability.
Starter flat rate (currently 2%) is replaced by per-category rates	Planned pricing change post-launch	Same timing rule as above applies — this is the first real occurrence of the general case, not a new one.	—

7. Rounding & minimum-fee cases

Case	Trigger	Expected behavior	Notes / open question
Commission rounds to a fraction of a piastre	% commission applied to a small basket total	Needs an explicit, documented rounding rule (e.g. round-half-up to the nearest piastre).	Without a locked rule, vendor statements and the platform's aggregate owed total can drift apart by rounding error across many orders.
Commission rounds to ~0 EGP on very small orders	Tiny order total × low starter rate	Not yet decided whether a minimum commission floor per order applies.	Business decision needed, not just a technical one.

8. Reporting & reconciliation

Case	Trigger	Expected behavior	Notes / open question
Owner needs total cash owed across all vendors	Admin dashboard view	Cross-vendor rollup of owed balances.	Doesn't exist yet — only per-vendor balances are modeled today.
Vendor needs their own statement / history	Vendor opens Earnings/wallet screen, or disputes a charge	Chronological per-order-item ledger: accrued, reversed, settled, with dates and references.	This is “Phase C” in the existing design — the Earnings screen is currently a stub.
Weekly settlement operational report	End of each settlement cycle	List of vendors: paid / partially paid / unpaid / paused, for whoever is doing collections.	Depends on §3's cycle-boundary decision being resolved first.

Framing note for backend

There are really two different sub-businesses hiding inside “commission wallet”, and they warrant different engineering treatment:

- **Self-delivery vendors (§1, §3, §4, §5)** — a genuine credit/collections problem: the platform extends trust, then has to track and chase repayment. This is where most of the open decisions above live.

- **Courier-delivered vendors** — not a debt at all; it's same-transaction cash netting handled entirely at the courier cash-handover layer. This mostly needs correct accounting (making sure these orders never touch the owed-balance ledger), not collections/enforcement machinery. See the companion *Buyer Refund Methodology* document for the one place courier-held cash creates a real open question.

Recommend backend scope this as: (1) lock the accrual state machine in §1 first since everything else depends on it, (2) resolve the settlement-cycle and payment-channel questions in §3 before building the settlement endpoint, (3) treat §2 and §5's identity-linking as a policy decision to raise with the owner, not something to silently default on.